

Index

Google.....	2
Sprinklr.....	6
Microsoft.....	8
Nvidia	13
Goldman Sachs	14
Adobe	17
Arista Networks.....	19
VISA	22
Willings INC.....	35
Cisco	36
Trilogy Innovations	39
HSBC.....	45
Samsung.....	49
InMobi Groups Pvt Ltd.....	50
Intel	51
Amazon.....	53

Google

1. Sort the Array

Description

You are given an integer array A of size N containing only **1**, **2**, and **3**. You are also given an integer Q and a 2D array of **queries**.

You have to answer Q queries:

- For the i^{th} query, perform $A[\text{queries}[i][0]] = \text{queries}[i][1]$ and determine the **minimum number of swaps** required to sort the array in non-decreasing order.

Task

For each query, determine the minimum swaps required to sort the array in non-decreasing order.

*Note: Array A and **queries** follow 0-based indexing.*

Example

Assumptions:

- $N = 5$
- $A = [1, 2, 3, 1, 2]$
- $Q = 2$
- **queries** = $[[1, 3], [3, 3]]$

Approach:

1. After the first update ($A[1] = 3$), the array A becomes **[1, 3, 3, 1, 2]**.

- Swap indices 1 and 3.
- Swap indices 2 and 4.

The minimum swaps required would be **2**.

2. After the second update, the array A becomes **[1, 3, 3, 3, 2]**.

- Swap indices 1 and 4.

The minimum swaps required is **1**.

2. Count Subsequences

Description

You are given an array A of N integer elements.

Determine the number of **non-empty subsequences** of array A such that there **do not exist three consecutive even or three consecutive odd integers** in the subsequence.

As the answer can be large, print it modulo $10^9 + 7$.

Notes

- Assume 1-based indexing.
- A subsequence of array A can be obtained by deleting some array elements while keeping the order of other elements intact.

Constraints

- $1 \leq T \leq 10$
- $1 \leq N \leq 10^5$
- $1 \leq A_i \leq 10^9$

Example

Assumptions:

- $N = 4$
- $A = [1, 2, 3, 5]$

Approach: The following subsequences satisfy the conditions:

- Length 1: $\{1\}, \{2\}, \{3\}, \{5\}$
- Length 2: $\{1, 2\}, \{1, 3\}, \{1, 5\}, \{2, 3\}, \{2, 5\}, \{3, 5\}$
- Length 3: $\{1, 2, 3\}, \{1, 2, 5\}, \{2, 3, 5\}$
- Length 4: $\{1, 2, 3, 5\}$

The subsequence $\{1, 3, 5\}$ **does not** satisfy the conditions as it has three consecutive odd integers.

Thus, the required answer is **14**.

3. Palindrome Special Number

Description

A number X is said to be **palindrome special** if it satisfies the following given conditions:

- Every digit d present in X occurs exactly d number of times in X .
- The digits of X form a palindrome (i.e., it reads the same from right to left and left to right).

You are given T test cases and for each test case, you are given an integer N .

Task

Determine the **smallest palindrome special number** strictly greater than N .

Constraints

- $1 \leq T \leq 10^5$
- $1 \leq N \leq 10^{17}$

Example

Assumptions:

- $N = 2$

Approach:

- **22** is the smallest palindrome special number greater than N .
- It contains the digit **2** exactly two times, and it is a palindrome.

4. Array Function

Description

You are given an array A of length N .

A function F is defined as follows:

$$F(A) = \sum_{i=1}^N (-1)^{(i+1)} A[i]$$

You also have an array B having M pairs where an element is in the form $[p, q]$ denoting that the elements of A at indices p and q are swappable. You can swap these indices any number of times and in any order.

Task

Determine the **maximum value** of $F(A)$ if you perform the swaps optimally.

Note: 1-based indexing is used.

Constraints

- $1 \leq T \leq 10^3$
- $1 \leq N, M \leq 10^5$
- $1 \leq A[i] \leq 10^9$
- $1 \leq B[i][0], B[i][1] \leq N$
- The sum of N over all test cases does not exceed 10^5 .
- The sum of M over all test cases does not exceed 10^5 .

Example

Assumptions:

- $N = 4$
- $A = [5, 6, 7, 8]$
- $M = 2$
- $B = [[1, 2], [3, 4]]$

Approach:

- The optimal approach will be to swap indices $[1, 2]$ and $[3, 4]$.
- The array after that will be $A = [6, 5, 8, 7]$.
- So answer is $F(A) = 6 - 5 + 8 - 7 = 2$.

Sprinklr

1. Gift Hamper Scam

Description

You are the organizer of a lucky draw contest where N participants are selected to receive gift hampers. You have K different types of gifts with at least 1 gift of each type.

You are tasked with distributing all the gifts among the N winners. However, the organizing committee puts a condition that you must distribute these gifts such that **each winner has all the gifts (he/she received) of the same type**.

Let's define **Scam Score** as the maximum number of gifts received by any winner with the above constraints on distribution. Print the **minimum possible Scam Score** for every test case.

Notes

- It is possible that a winner might not get any gift.

Constraints

- $1 \leq T \leq 10$
- $1 \leq N \leq 10^9$
- $1 \leq K \leq 10^5$
- $K \leq N$
- $1 \leq \text{Number of gifts available for any type} \leq 10^9$

Example

Assumptions: $N = 7, K = 5$, Gifts Array = [7, 1, 7, 4, 4].

Approach: Types 1 to 5 have quantities: 7, 1, 7, 4, 4 respectively. An optimal distribution is:

- Winner 1: 4 gifts of Type 1
- Winner 2: 3 gifts of Type 1
- Winner 3: 1 gift of Type 2
- Winner 4: 3 gifts of Type 3
- Winner 5: 4 gifts of Type 3
- Winner 6: 4 gifts of Type 4
- Winner 7: 4 gifts of Type 5

The maximum gifts received by any player is **4** (Winners 1, 5, 6, 7). Thus, Scam Score = **4**.

2. Gambler's Dice

Description

You are given an n -ary perfect tree with k levels. You need to mark each node of this tree with a number on a dice such that the resulting tree is Gambler Friendly.

A tree is **Gambler Friendly** if the following holds:

- For every node of the tree, let say it is marked with a number p ($1 \leq p \leq 6$), then every adjacent node of this tree is marked with a number q that is adjacent to p on the dice.
- (i.e., p and q are adjacent on the dice and $p \neq q$).

You need to calculate the number of possible ways to paint the tree such that it is Gambler Friendly. Since the number can be very large, print it modulo $10^9 + 7$.

Notes

- A number is not adjacent to itself on a dice.
- Input Format: 2 space separated integers, n and k .
- Reference: A perfect binary tree is a tree in which all interior nodes have two children and all leaves have the same depth.

Constraints

- $2 \leq n \leq 10^5$
- $1 \leq k \leq 10^5$

Example

Sample Case

Input:

3 3

Output:

100663296

3. Auto Suggest

Description

Given a dictionary consisting of N words, and a query word S , auto-suggest the best word from the dictionary that is closest to S .

Two words are first compared by their **Levenshtein distance** to the word S . If two words have the same Levenshtein distance, then the **lexicographically smaller** word is given priority.

Task: Auto-suggest the closest word to S from the given dictionary of N words.

Notes

- **Levenshtein distance** between two strings is defined as the minimum number of edits required to obtain one string from the other.

- An "edit" is defined by either an insertion of a character, a deletion of a character, or a replacement of a character.
- All the words and the query word S consist of lowercase alphabets only.

Constraints

- $1 \leq N \leq 100$
- $1 \leq \text{words}[i].\text{length} \leq 10$
- $1 \leq S.\text{length} \leq 10$

Example

Assumptions:

- $N = 5$
- $\text{words} = [\text{"tocor"}, \text{"torect"}, \text{"tocfrrec"}, \text{"tocorre"}, \text{"toco fecd"}]$
- $S = \text{"tocorrect"}$

Approach: The strings belonging to the dictionary having a minimum Levenshtein distance of **2** from the given string **"tocorrect"** are [**"tocfrrec"**, **"tocorre"**].

Among the two, **"tocfrrec"** is lexicographically smaller than **"tocorre"**. So, the answer is **"tocfrrec"**.

Microsoft

1. Discounted Purchase

Description

Alex has a list of items to purchase at a market. The owner offers to discount each item after the first one by the **lowest marked price among the prior items**.

No item's price can be discounted below **0**, and the list of items may not be reordered. Calculate the total payable amount.

Task: Complete the function **calculateAmount** which returns Alex's total cost to purchase all the items.

Notes

- The first item is never discounted.
- The minimum cost of any item is **0** (prices cannot be negative).

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq \text{prices}[i] \leq 10^7$, where $0 \leq i < n$

Example

Assumptions:

- **prices** = [2, 5, 1, 4]

Approach:

- **1st Item:** Pays **2** (no previous items). Min previous = **2**.
- **2nd Item:** Costs $5 - 2 = 3$. New Min previous = $\min(2, 5) = 2$.
- **3rd Item:** Costs $1 - 2 = -1 \rightarrow 0$ (since cost ≥ 0). New Min previous = $\min(2, 1) = 1$.
- **4th Item:** Costs $4 - 1 = 3$.

Total Cost = $2 + 3 + 0 + 3 = 8$.

2. Palindrome Subsequences

Description

For a string s consisting only of characters '0' and '1', determine the number of **subsequences of length 5** that are palindromes.

Since the result can be very large, return the answer modulo $10^9 + 7$.

Notes

- Two subsequences are considered different if the **indices** used to form them are different.

Constraints

- $1 \leq |s| \leq 10^4$
- The string s contains only characters '0' and '1'.

Example

Assumptions: $s = "0100110"$

Approach: We need to find subsequences of length 5 that read the same forwards and backwards. Using 1-based indexing, there are exactly **5** such subsequences:

- Indices (1, 2, 3, 5, 7) $\rightarrow$ chars **0, 1, 0, 1, 0** (Palindrome)
- Indices (1, 2, 3, 6, 7) $\rightarrow$ chars **0, 1, 0, 1, 0** (Palindrome)
- Indices (1, 2, 4, 5, 7) $\rightarrow$ chars **0, 1, 0, 1, 0** (Palindrome)
- Indices (1, 2, 4, 6, 7) $\rightarrow$ chars **0, 1, 0, 1, 0** (Palindrome)
- Indices (1, 2, 5, 6, 7) $\rightarrow$ chars **0, 1, 1, 1, 0** (Palindrome)

Total count = 5.

3. Game of Stones

Description

Alice and Bob continue their games with stones. There is a row of n stones, and each stone has an associated value. You are given an integer array `stones`, where `stones[i]` is the value of the i^{th} stone.

Alice and Bob take turns, with Alice starting first. On each turn, the player may remove any stone from `stones`. The player who removes a stone loses if the sum of the values of all removed stones is divisible by 3. Bob will win automatically if there are no remaining stones (even if it is Alice's turn).

Assuming both players play optimally, return **true** if Alice wins and **false** if Bob wins.

Constraints

- $1 \leq \text{stones}[i] \leq 10^4$
- $1 \leq \text{stones.length} \leq 10^5$.

Example

Assumptions: $stones = [2, 1]$

Approach: The game will be played as follows:

- Turn 1: Alice can remove either stone.
- Turn 2: Bob removes the remaining stone. The sum of the removed stones is $1 + 2 = 3$ and is divisible by 3. Therefore, Bob loses and Alice wins the game.

Output: true

4. Text Editor Prototype

Description

Implement a prototype for a text editor application with the following functionalities. You are given a 2-D array of strings of size $n \times 2$ named **commands**. Start with an empty string and the cursor at the starting position (index 0).

Command	Action
["Insert", s]	Insert the string s at the current cursor position. The cursor will also move to the right by the length of the inserted string.
["Print", x]	If the cursor is at position m , print all the characters from $m - x$ to $m + x$ (inclusive).
["Left", x]	Move the cursor x positions to the left.
["Right", x]	Move the cursor x positions to the right.
["Backspace", x]	Remove x characters to the left of the cursor.
["Delete", x]	Remove x characters to the right of the cursor.

Task: Find the string obtained on each of the print operations by performing the above operations starting from an empty string and cursor at the starting position. Report them as an array of printed strings in order.

Notes

- If for any operation the cursor reaches the beginning or the end of the current string, it does not go any further on either side.
- For consistency, the second argument is always provided as a string and hence might need parsing (e.g., **"5"** instead of integer **5**).

Constraints

- $1 \leq n \leq 5000$
- The sum of lengths of all strings inserted does not exceed 5×10^5 .
- The size of the first string inserted does not exceed 100 and the length of all other strings inserted does not exceed 25.
- All the strings consist of lowercase English letters.

Example

Assumptions:

- `commands = ["Insert", "hello"], ["Left", "2"], ["Print", "1"]`

Approach: 1. `["Insert", "hello"]`: Text becomes `"hello"`. Cursor moves to index 5. 2. `["Left", "2"]`: Cursor moves left by 2 positions. New cursor $m = 3$. 3. `["Print", "1"]`: Print characters from index $3 - 1$ to $3 + 1$ (indices 2 to 4).

- Characters at indices 2, 3, 4 are `'l'`, `'l'`, `'o'`.

Output string is `"llo"`.

5. Perfect Substring

Description

A string s , comprised of digits from **0** to **9**, contains a **perfect substring** if all characters within that substring appear exactly k times.

Determine the total number of perfect substrings in s .

Task: Complete the function **perfectSubstring** which takes the string s and integer k as arguments and returns the count of perfect substrings.

Notes

- 0-based indexing is used.

Constraints

- $1 \leq |s| \leq 10^5$
- $0 \leq s[i] \leq 9$ (where $0 \leq i < |s|$)
- $1 \leq k \leq 10^5$

Example

Assumptions:

- $s = "1102021222"$
- $k = 2$

Approach: The following **6** substrings are perfect (every digit inside them appears exactly $k = 2$ times):

1. $s[0 : 1] = "11"$ (Digit `'1'` appears 2 times)
2. $s[0 : 5] = "110202"$ (Digits `'1'`, `'0'`, `'2'` each appear 2 times)
3. $s[1 : 6] = "102021"$
4. $s[2 : 5] = "0202"$
5. $s[7 : 8] = "22"$
6. $s[8 : 9] = "22"$

6. Merge Intervals

Description

You are given an array of intervals, where each interval is represented as a pair

$$\mathbf{intervals}[i] = [\mathbf{start}_i, \mathbf{end}_i].$$

Two intervals are said to **overlap** if they share at least one common point. Your task is to merge all overlapping intervals and return an array of **non-overlapping intervals** that together cover all intervals in the input.

Task

Given the list of intervals **intervals**, merge all overlapping intervals and return the resulting list of merged intervals.

Constraints

- $1 \leq \mathbf{intervals.length} \leq 10^4$
- $\mathbf{intervals}[i].length = 2$
- $0 \leq \mathbf{start}_i \leq \mathbf{end}_i \leq 10^4$

Example

Input:

$$\mathbf{intervals} = [[1, 3], [2, 6], [8, 10], [15, 18]]$$

Explanation:

- The intervals **[1,3]** and **[2,6]** overlap, so they are merged into **[1,6]**.
- The interval **[8,10]** does not overlap with any other interval.
- The interval **[15,18]** does not overlap with any other interval.

Output:

$$[[1, 6], [8, 10], [15, 18]]$$

Nvidia

Note: Solve the following problems in C language.

1. FizzBuzz

Description

Write a short program that prints each number from 1 to 100 on a new line.

For each multiple of 3, print "Fizz" instead of the number.

For each multiple of 5, print "Buzz" instead of the number.

For numbers which are multiples of both 3 and 5, print "FizzBuzz" instead of the number.

2. Palindrome Check

Description

You are provided with a partial C program designed to check if a given number is a **palindrome**. A number is a palindrome if it reads the same backward as forward (e.g., **121**, **44**, **12321**).

Task: Complete the function **isPalindrome** in the editor. It must accept an integer **N** and return **1** if the number is a palindrome, or **0** otherwise.

Code Snippet

C Language Source

```
#include <stdio.h>

int isPalindrome(int N) {
    // Write your code here
}

int main() {
    int N = 121;
    if (isPalindrome(N)) {
        printf("Yes\n");
    }
    else {
        printf("No\n");
    }
    return 0;
}
```

Goldman Sachs

1. Unique Co-prime Pairs

Description

Write a program that accepts a number N as input and prints the number of **unique co-prime pairs** where both numbers are less than N .

Two numbers are considered co-prime if their greatest common divisor (GCD) is 1.

Notes

- Read the input from **STDIN** and print the output to **STDOUT**.
- Do not print arbitrary strings anywhere in the program, as these contribute to the output and test cases will fail.
- Based on the example, a "unique pair" (a, b) implies $a < b < N$.

Constraints

- $3 \leq N \leq 9,999$

Example

Assumptions: $N = 10$

Approach: We need to find pairs (a, b) such that $a < b < 10$ and $\gcd(a, b) = 1$. There are **27** such unique pairs:

- $(1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (1, 8), (1, 9)$
- $(2, 3), (2, 5), (2, 7), (2, 9)$
- $(3, 4), (3, 5), (3, 7), (3, 8)$
- $(4, 5), (4, 7), (4, 9)$
- $(5, 6), (5, 7), (5, 8), (5, 9)$
- $(6, 7)$
- $(7, 8), (7, 9)$
- $(8, 9)$

Total count is 27.

2. Inventory Scan

Description

You are developing an evaluation system for a warehouse robot that scans inventory racks arranged in a rectangular grid. Each rack in the grid contains a certain number of inventory

units.

The robot is programmed to follow a ****spiral path**** starting from the top-left corner of the grid, moving clockwise around the perimeter, and then moving inward layer by layer, scanning each rack until all racks are covered.

Task: Write a program that takes the dimensions of the grid (N rows and M columns) and the inventory units in each rack as input, and determines the inventory units reading obtained by the robot using the spiral path.

Notes

- Read the input from **STDIN** and print the output to **STDOUT**.
- Do not print arbitrary strings anywhere in the program, as these contribute to the output and test cases will fail.
- The path starts at $(0,0)$ and moves **Right** $\rightarrow$ **Down** $\rightarrow$ **Left** $\rightarrow$ **Up**, repeating inwards.

Constraints

- $1 \leq N, M \leq 100$

Example

Assumptions:

- $N = 3, M = 2$
- Grid: $\begin{bmatrix} 10 & 3 \\ 8 & 5 \\ 12 & 14 \end{bmatrix}$

Approach: The robot scans the racks in the following spiral order:

1. Start Top-Left: **10**
2. Move Right: **3**
3. Move Down (Right edge): **5** $\rightarrow$ **14**
4. Move Left (Bottom edge): **12**
5. Move Up (Left edge): **8**

Final Sequence: **10 3 5 14 12 8**.

3. Pencil Box Nesting

Description

Dwarak owns N transparent pencil boxes, each with a fixed hollow space starting from a certain length and ending at another — represented by two numbers:

- L : the left edge (start of hollow space)
- R : the right edge (end of hollow space), where $L < R$.

The boxes are fixed in a rectangular shape and cannot be reshaped or repositioned. Dwarak is trying to fit the pencil boxes one inside another, without moving or rotating them.

Nesting Condition: For a box A to be nested inside another box B :

- A must have the same or **larger** left edge (L) than B , AND
- A must have the same or **smaller** right edge (R) than B .

(i.e., If B is $[5, 19]$, then for A to nest inside B , A 's left edge has to be 5 or more, and right edge has to be 19 or less).

Task: Find the **longest possible nesting sequence** of pencil boxes — from outermost to innermost. Print the maximum number of boxes that can be nested, along with their stacking order.

Notes

- If multiple such sequences are possible, then pick the stack whose **left edge number value is more**.
- Read input from **STDIN** and print output to **STDOUT**.
- The output should first print the length of the longest sequence (K), followed by K lines containing the L and R of the boxes in stacking order (outermost to innermost).

Constraints

- $1 \leq N \leq 10000$
- $1 \leq L < R \leq 10^3$

Example

Assumptions:

- $N = 3$
- Boxes: $[4, 5]$, $[3, 6]$, $[2, 7]$

Approach:

- Box C (2 to 7) can hold Box B (3 to 6) because $2 \leq 3$ and $6 \leq 7$.
- Box B (3 to 6) can hold Box A (4 to 5) because $3 \leq 4$ and $5 \leq 6$.

The sequence is $C \rightarrow B \rightarrow A$. Total length is 3.

Sample Case 1

Sample Input:

```
3
4 5
3 6
2 7
```

Sample Output:

```
3
2 7
3 6
4 5
```

Adobe

1. White Light

Description

The OpenLand Film and Theater Society is organizing a street play. The stage is n meters long and has m lights installed above it. Each light can be red, blue, or green.

When all three colors shine on a particular area, it appears **white**. Each light illuminates a specific range on the stage, defined by three attributes: **color**, **left**, and **right**.

- The **color** attribute is represented by **1** (Red), **2** (Blue), or **3** (Green).
- The **left** and **right** attributes represent the starting and ending points of the illuminated range (inclusive).

Task: Determine the total number of integer positions on the stage that are illuminated with white light (i.e., covered by at least one Red, one Blue, and one Green light).

Function Description

- **n**: The length of the stage.
- **m**: The number of lights.
- **lights**: A 2D array where each row is [**color**, **left**, **right**].

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 10^5$
- $1 \leq \text{left} \leq \text{right} \leq n$
- $\text{color} \in \{1, 2, 3\}$

Example

Assumptions:

- $n = 5$
- $m = 3$
- **lights** = $[[1, 1, 5], [2, 2, 5], [3, 3, 5]]$

Approach:

- **Light 1 (Red):** Covers range $[1, 5]$.
- **Light 2 (Blue):** Covers range $[2, 5]$.
- **Light 3 (Green):** Covers range $[3, 5]$.

The integer positions covered by all three colors are **3**, **4**, and **5**. Total count = **3**.

Sample Case

Sample Input:

```
5 3
1 1 5
2 2 5
3 3 5
```

Sample Output:

```
3
```

2. Special Binary String

Description

Special binary strings are binary strings with the following two properties:

1. The number of '**0**'s is equal to the number of '**1**'s.
2. Every prefix of the binary string has at least as many '**1**'s as '**0**'s.

You are given a special binary string s .

A **move** consists of choosing two consecutive, non-empty, special substrings of s , and swapping them. (Two strings are consecutive if the last character of the first string is exactly one index before the first character of the second string).

Task: Return the **lexicographically largest** resulting string possible after applying the mentioned operations any number of times.

Constraints

- $1 \leq |s| \leq 50$
- $s[i]$ is either '**0**' or '**1**'.
- s is guaranteed to be a special binary string.

Example

Assumptions: $s = "11011000"$.

Approach:

- Identify special substrings: "**10**" (at index 1) and "**1100**" (at index 3).
- These are consecutive. Swapping them results in "**11100100**".
- This is the lexicographically largest string achievable.

Sample Case 1

Sample Input:

```
11011000
```

Sample Output:

```
11100100
```

Arista Networks :)

Note: The followings questions are not exact but same concepts were involved in the original problems.

1. Lexicographically Smallest Subsequence

Description

You are given a string s consisting of lowercase English letters (ASCII characters). You are also given an array of strings $A = [a_1, a_2, \dots, a_k]$.

Your task is to find a string from the array A that satisfies the following two conditions:

1. It is a **subsequence** of the string s .
2. It is the **lexicographically smallest** among all strings in A that satisfy condition 1.

If there are multiple valid strings that are lexicographically smallest, return any one of them. If no string in A is a subsequence of s , return "-1" (as a string).

Input Format

- The first line contains the string s .
- The second line contains an integer k , the size of the array A .
- The next k lines each contain a string a_i , representing the elements of the array.

Output Format

Print the lexicographically smallest string from A that is a subsequence of s . If no such string exists, print -1.

Constraints

- $1 \leq |s| \leq 10^5$
- $1 \leq k \leq 1000$
- $1 \leq |a_i| \leq 1000$
- All strings consist of lowercase English letters.

Example

Input:

```
abppplee
4
able
ale
apple
```

bale

Output:

able

Explanation: The strings "able", "ale", and "apple" are subsequences of "abppplee". "bale" is not. Among the valid subsequences, "able" is the lexicographically smallest.

2. Subarrays with K Different Integers

Description

Given an integer array **nums** and an integer **k**, return the number of **good subarrays** of **nums**.

A **good array** is an array where the number of different integers in that array is exactly **k**.

- For example, **[1,2,3,1,2]** has **3** different integers: **1**, **2**, and **3**.

A **subarray** is a **contiguous** part of an array.

Constraints

- $1 \leq \text{nums.length} \leq 2 \times 10^4$
- $1 \leq \text{nums}[i], k \leq \text{nums.length}$

Example

Assumptions:

- $\text{nums} = [1, 2, 1, 2, 3]$
- $k = 2$

Approach: We need to find subarrays formed with exactly **2** different integers. The **7** valid subarrays are:

- **[1, 2]**
- **[2, 1]**
- **[1, 2]**
- **[2, 3]**
- **[1, 2, 1]**
- **[2, 1, 2]**
- **[1, 2, 1, 2]**

Sample Case 1

Sample Input:

$\text{nums} = [1, 2, 1, 2, 3]$

$k = 2$

Sample Output:

7

3. Maximum Sum Combinations

Description

You are given two integer arrays a and b of equal size. A **sum combination** is formed by adding one element from a and one from b , using each index pair (i, j) at most once.

Task: Return the **top k maximum sum combinations**, sorted in non-increasing order.

Constraints

- $1 \leq a.size() = b.size() \leq 10^5$
- $1 \leq k \leq a.size()$
- $1 \leq a[i], b[i] \leq 10^4$

Example

Assumptions:

- $a = [3, 2]$
- $b = [1, 4]$
- $k = 2$

Approach: The possible sums from all pairs are:

- $3 + 1 = 4$
- $3 + 4 = 7$
- $2 + 1 = 3$
- $2 + 4 = 6$

The list of all sums is $[4, 7, 3, 6]$. Sorting them gives $[7, 6, 4, 3]$. The top **2** sums are **7** and **6**.

Sample Case 1

Sample Input:

$a = [3, 2]$

$b = [1, 4]$

$k = 2$

Sample Output:

$[7, 6]$

VISA

1. Access Code Fragments

Description

A spy agency has intercepted fragments of a secret code. Each fragment is represented as a positive integer in an array called **fragments**. Intelligence suggests that the complete access code is a specific number called **accessCode**.

Your mission is to determine **how many different ways the fragments can be paired** to form the complete access code.

The agency's cryptography rules state that:

- When fragments are combined, they are simply placed next to each other (concatenated, not added mathematically).
- For example, if fragment "12" is followed by fragment "34", the result is "1234".

Notes

- Each specific pair of fragments (by position in the array) counts as a unique combination.
- The fragments can be used in any order, but each fragment position can only be used once in a combination.
- You are looking for pairs (i, j) such that $i \neq j$ and $\text{concat}(\text{fragments}[i], \text{fragments}[j]) == \text{accessCode}$.

Constraints

- $2 \leq \text{fragments.length} \leq 3 \times 10^4$
- $1 \leq \text{fragments}[i] \leq 10^9$
- $1 \leq \text{accessCode} \leq 10^9$

Example

Assumptions:

- **fragments** = [1, 212, 12, 12]
- **accessCode** = 1212

Approach: We can form the access code **1212** by combining:

- **fragments**[0] (1) with **fragments**[1] (212) $\rightarrow$ 1212
- **fragments**[2] (12) with **fragments**[3] (12) $\rightarrow$ 1212
- **fragments**[3] (12) with **fragments**[2] (12) $\rightarrow$ 1212

Note that the last two combinations are considered different because they use different positions in the array.

Total count = 3.

2. Aligned Memory Allocator

Description

You are given an array of integers **memory** consisting of 0s and 1s which indicates whether the corresponding memory unit is free or not.

- **memory[i] = 0** means that the i^{th} memory unit is free.
- **memory[i] = 1** means it is occupied.

The memory is aligned with segments of 8 units, so all occupied memory blocks must start at an index divisible by 8 (e.g., 0, 8, 16, etc).

Your task is to perform two types of queries:

1. **alloc x**: Find the left-most **aligned** memory block of **x** consecutive free memory units and mark these units as occupied.
 - (i.e., find the left-most contiguous subarray of 0s, starting at the position **start** which is divisible by 8, and replace all these memory units with 1s).
 - If there is no proper aligned memory block with **x** consecutive free units, return **-1**.
 - Otherwise, return the index of the first position of the allocated block segment and assign an **ID** to every single element in the block, based on an **atomic counter** (the counter starts at 1 and is incremented on every successful **alloc** operation).
2. **erase ID**: If there exists an allocated memory block with element ids equal to **ID**, free all its memory units (set all of the bits in the block to 0).
 - Return the length of the deleted memory block.
 - If there is no such **ID** or the block with this **ID** has already been deleted, return **-1**.

Task: Return an array containing the results of all the queries.

Notes

- **x** may be greater than 8, so the block may cover more than one memory segment.
- **queries** is an array of 2-element arrays.
- If **queries[i][0] = 0**, then this is an **alloc** type query, where **x = queries[i][1]**.
- If **queries[i][0] = 1**, then this is an **erase** type query, where **ID = queries[i][1]**.
- You are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(\text{queries.length} \times \text{memory.length})$.

Constraints

- $1 \leq \text{memory.length} \leq 10^5$
- $1 \leq \text{queries.length} \leq 10^5$

Example

Assumptions:

- **memory** = [0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1]
- **queries** = [[0, 2], [0, 1], [0, 1], [1, 1], [0, 3], [1, 4], [0, 4]]

Approach:

1. **[0, 2]** (alloc 2): Finds aligned slot at index 8. ID=1 assigned. Returns **8**.
2. **[0, 1]** (alloc 1): Finds aligned slot at index 0 (slot 0 is 0, but slot 1 is 1... wait, alignment rule: starts at 0, 8, etc. Index 0 has length? No, index 0 is 0, index 1 is 1. Block at 0 is obstructed. Next aligned is 8 (occupied). Next is 16...). *(See explanation in image: "allocates a memory block from units 8 to 9, as 8 is the first aligned position with 2 free slots.")*
3. Result array: [8, 0, -1, 2, 8, -1, -1].

Sample Case 0

Sample Input:

memory = [0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1]

queries = [[0, 2], [0, 1], [0, 1], [1, 1], [0, 3], [1, 4], [0, 4]]

Sample Output:

[8, 0, -1, 2, 8, -1, -1]

3. Odd Zero Counter

Description

You are given an array of non-negative integers a . Your task is to calculate how many elements of a have an odd number of occurrences of the digit **0**.

For example, for input **[4, 50, 100, 65, 2000, 700, 1, 10]**, the answer should be **3**. (These numbers are **50**, **2000**, and **10**).

Task: Calculate and return the count of such elements.

Notes

- You are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(a.length^2)$ will fit within the execution time limit.
- Remember: 0 occurrences is an **even** number.

Constraints

- $1 \leq a.length \leq 10^3$
- $0 \leq a[i] \leq 10^9$

Example

Assumptions:

- $a = [20, 11, 10, 10070, 7]$

Approach: We analyze the occurrence of the digit **0** in each element:

- $a[1] = 11$ and $a[4] = 7$: The digit **0** does not occur (0 times), so these do not count (0 is even).
- $a[0] = 20$ and $a[2] = 10$: The digit **0** occurs **1 time** (odd), so these count.
- $a[3] = 10070$: The digit **0** occurs **3 times** (odd), so this element counts.

Total count = **3**.

Sample Case 0

Input:

$a = [20, 11, 10, 10070, 7]$

Output:

3

4. Resource Processing

Description

You are given an array of **resources**, with each being an **"A"** or **"P"**. All **"A"**s are arranged at the beginning of the array and **"P"**s stand at the end.

A process is executed on this array until it halts. You are given a variable **conversionRate**, which represents the number of **"P"** resources that can be deleted and exchanged for one **"A"** resource.

Task: Follow the process and compute how many cycles will pass until the process halts.

Cycle Events: Each cycle, one of the three events happens (checked in this order):

1. **Option 1:** If there are at least **conversionRate** **"P"** resources, then the last **conversionRate** **"P"** resources are removed and one **"A"** is added at the **beginning** of the array.
2. **Option 2:** If there is at least one **"A"**, the last **"A"** changes to **"P"**.
3. **Option 3:** If neither Option 1 nor Option 2 can be completed, then the process halts.

Notes

- You are not expected to provide the most optimal solution.
- A solution with time complexity not worse than $O(\text{resources.length} \times \text{conversionRate}^2)$ will fit within the execution time limit.

Example

Assumptions:

- **resources** = ["A", "A", "A", "P", "P", "P"]
- **conversionRate** = 2

Approach:

- **Start:** ["A", "A", "A", "P", "P", "P"]
- **Cycle 1 (Opt 1):** 3 Ps $\geq$ 2. Remove 2 Ps, Add A. $\rightarrow$ ["A", "A", "A", "A", "P"]

- **Cycle 2 (Opt 2):** Last A becomes P. $\rightarrow$ ["A", "A", "A", "P", "P"]
- **Cycle 3 (Opt 1):** 2 Ps $\geq$ 2. Remove 2 Ps, Add A. $\rightarrow$ ["A", "A", "A", "A"]
- **Cycle 4 (Opt 2):** Last A becomes P. $\rightarrow$ ["A", "A", "A", "P"]
- **Cycle 5 (Opt 2):** Last A becomes P. $\rightarrow$ ["A", "A", "P", "P"]
- **Cycle 6 (Opt 1):** Remove 2 Ps, Add A. $\rightarrow$ ["A", "A", "A"]
- ... (Process continues) ...
- **Total Cycles: 13**

Sample Case 1

Input:

resources = ["A", "A", "A", "P", "P", "P"]

conversionRate = 2

Output:

13

5. Network Device Status

Description

You are monitoring the performance of devices in a network. Each device periodically reports its status by sending a numerical value. These values are collected in sequence in an array.

Given an array of integers **statusReports**, your task is to find the **sum of all elements** in the array **statusReports** that are ****greater than their immediate neighbors****.

Boundary Condition: If an element is the first or last in the array, it should be compared only to its single neighbor.

Task: Return the resulting sum as an integer.

Notes

- A solution with time complexity not worse than $O(\text{statusReports.length}^2)$ will fit within the execution time limit.

Constraints

- $1 \leq \text{statusReports.length} \leq 2000$
- $0 \leq \text{statusReports}[i] \leq 10^9$

Example

Assumptions:

- **statusReports** = [1, 3, 2, 5, 1]

Approach: We iterate through the array and compare each element with its neighbors:

- **Index 0 (Value 1):** Neighbor is 3. $1 \not> 3$. (Skip)
- **Index 1 (Value 3):** Neighbors are 1 and 2. $3 > 1$ AND $3 > 2$. (Add 3)

- **Index 2 (Value 2):** Neighbors are 3 and 5. $2 \not> 3$. (Skip)
- **Index 3 (Value 5):** Neighbors are 2 and 1. $5 > 2$ AND $5 > 1$. (Add 5)
- **Index 4 (Value 1):** Neighbor is 5. $1 \not> 5$. (Skip)

Total Sum = $3 + 5 = 8$.

Sample Case 0

Input:

statusReports = [1, 3, 2, 5, 1]

Output:

8

6. Space Station Shuttle

Description

A space station network has two major hubs, **Alpha** and **Beta**, connected by shuttle services.

- Shuttles from **Alpha** to **Beta** take **100** time units and depart according to a sorted integer array **alpha2beta**.
- Similarly, shuttles from **Beta** to **Alpha** also take **100** time units and depart at times specified in a sorted integer array **beta2alpha**.

You need to complete **missions** where each mission requires traveling from *Alpha* to *Beta* and back to *Alpha*. Calculate the time unit when you will complete all missions, assuming you always take the **earliest available shuttle**.

Task: Return the time unit when the last mission is completed.

Notes

- It is guaranteed that it is possible to complete all missions using the provided shuttle schedules.
- All times are measured in time units from the start of your journey (time **0**).
- You are not expected to provide the most optimal solution.

Constraints

- $1 \leq \text{alpha2beta.length}, \text{beta2alpha.length} \leq 100$
- $1 \leq \text{missions} \leq 100$
- Travel time is fixed at **100** units each way.

Example

Assumptions:

- **alpha2beta** = [109, 200, 500]
- **beta2alpha** = [99, 210, 600]
- **missions** = 2

Approach:

1. Mission 1:

- Depart *Alpha* at **109**. Arrive *Beta* at $109 + 100 = \mathbf{209}$.
- Earliest return from *Beta* ≥ 209 is **210**.
- Depart *Beta* at **210**. Arrive *Alpha* at $210 + 100 = \mathbf{310}$.

2. Mission 2:

- Current time is **310**. Earliest departure from *Alpha* ≥ 310 is **500**.
- Depart *Alpha* at **500**. Arrive *Beta* at $500 + 100 = \mathbf{600}$.
- Earliest return from *Beta* ≥ 600 is **600**.
- Depart *Beta* at **600**. Arrive *Alpha* at $600 + 100 = \mathbf{700}$.

Total completion time is **700**.

Sample Case 1

Input:

alpha2beta = [109, 200, 500]

beta2alpha = [99, 210, 600]

missions = 2

Output:

700

7. Count Power Readings

Description

You are monitoring energy usage in a smart grid system and need to identify specific data patterns in the readings.

You are given an array of integers **readings** representing the energy readings over a period, and an integer **k**. Your task is to count how many readings are powers of **k**.

In this context, a **power of k** refers to numbers that can be expressed as:

$$k^0, k^1, k^2, \dots$$

Return the total number of elements in **readings** that are powers of **k**.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than

$$O(\text{readings.length}^2)$$

will fit within the execution time limit.

Input

- **readings**: An array of integers representing the energy readings over a period.
- **k**: An integer representing the base of the power to check against the readings.

Constraints

- $1 \leq \text{readings.length} \leq 1000$
- $2 \leq \text{readings}[i] \leq 10^9$
- $2 \leq k \leq 10^9$

Example

Input:

readings = [2, 4, 7, 8, 16, 32, 120], **k** = 2

Explanation: The numbers in **readings** that are powers of 2 are:

$$2 = 2^1, 4 = 2^2, 8 = 2^3, 16 = 2^4, 32 = 2^5$$

There are 5 such numbers.

Output:

solution(readings, k) = 5

Input:

readings = [10201, 101, 1030301, 101, 1011], **k** = 101

Explanation: The numbers in **readings** that are powers of 101 are:

$$101 = 101^1, 10201 = 101^2, 1030301 = 101^3$$

There are 3 such numbers.

Output:

solution(readings, k) = 3

8. Explode Bubbles

Description

You are given a board of cells, each containing a bubble of a specific color. Neighboring cells of the bubbles are defined as adjacent cells in the top, bottom, left, and right directions.

Task: Your task is to perform a bubble explosion on the board. A bubble explosion is defined by the following rules:

- A bubble within any given cell is eligible to explode **if it has the same color as bubbles in at least 2 neighboring cells**.
- All eligible bubbles and bubbles of the same color in their neighboring cells are marked for explosion.
- All marked bubbles explode at the same time. Exploded bubbles are removed from the board, resulting in empty cells (represented by **0**).
- After all exploded bubbles are removed, remaining bubbles in cells above the empty cells drop down to fill all empty cells.

You are given an initial board of cells **bubbles**—a multidimensional array of integers representing cells containing bubbles of various colors. Return the board state after a bubble explosion.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than $O(\text{bubbles.length}^2 \cdot \text{bubbles}[0].\text{length}^2)$ will fit.

Constraints

- $1 \leq \text{bubbles.length} \leq 100$
- $1 \leq \text{bubbles}[0].\text{length} \leq 100$
- $1 \leq \text{bubbles}[i][j] \leq 10^4$

Example

Input:

$$\text{bubbles} = \begin{bmatrix} 3 & 1 & 2 & 1 \\ 1 & 1 & 1 & 4 \\ 3 & 1 & 2 & 2 \\ 3 & 3 & 3 & 4 \end{bmatrix}$$

Approach & Explanation:

1. Identify Eligible Bubbles:

- The **1** at (1, 1) has neighbors (0, 1), (1, 0), (1, 2), and (2, 1) which are all **1**s. Since it has ≥ 2 neighbors of the same color, it is eligible. It marks itself and all its neighbors for explosion.
- The **3** at (3, 0) has neighbors (2, 0) and (3, 1) which are **3**s. It is eligible and marks neighbors (2, 0) and (3, 1).
- The **3** at (3, 1) has neighbors (3, 0) and (3, 2) which are **3**s. It is eligible and marks neighbors.

2. Explosion: All marked bubbles are removed (set to **0**).

- The cross of **1**s explodes.
- The bottom-left cluster of **3**s explodes. Note that the top-left **3** at (0, 0) has no matching neighbors, so it survives.

$$\text{After Blast (Before Gravity)} = \begin{bmatrix} 3 & 0 & 2 & 1 \\ 0 & 0 & 0 & 4 \\ 0 & 0 & 2 & 2 \\ 0 & 0 & 0 & 4 \end{bmatrix}$$

3. Gravity: Remaining bubbles fall down to fill empty slots.

- Col 0: The single **3** falls to the bottom.
- Col 2: The two **2**s fall to the bottom.

Output:

$$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 4 \\ 0 & 0 & 2 & 2 \\ 3 & 0 & 2 & 4 \end{bmatrix}$$

9. Newspaper Page Formatting

Description

Imagine that you are formatting the text that will appear on a newspaper page. The text is provided to you in the following format:

- **paragraphs** is an array of paragraphs, where each paragraph itself is represented by an array containing chunks of text.
- **width** represents the maximum number of characters that can fit on each line of the newspaper page.

Your task is to produce a formatted newspaper page according to the specifications described below and return it as an array of strings, where the i^{th} string represents the i^{th} line of the newspaper page.

Formatting Rules

- For each paragraph **paragraphs[i]**, start formatting on a new line and add the text chunks **paragraphs[i][j]** in order.
- When multiple chunks of text appear on the same line, they must be separated by exactly one space.
- You are not allowed to break a chunk of text. If adding the next chunk (including the separating space) would cause the line length to exceed **width**, start a new line and place the chunk there.
- It is possible that a line contains fewer characters than **width**. In such cases, align the text to the center by adding spaces around it:
 - If the remaining space is even, add an equal number of spaces before and after the text.
 - If the remaining space is odd, add an equal number of spaces on both sides, and place the extra space **after** the text.
- Add a rectangular border of asterisks (*) around the entire newspaper page:
 - The top and bottom borders consist of **width + 4** asterisks.
 - Each text line is surrounded by * on the left and * on the right.
 - Border characters do not count toward the **width**.

Note: You are not expected to provide the most optimal solution, but a solution with time complexity not worse than

$$O(\text{paragraphs.length} \cdot \text{paragraphs}[0].\text{length} \cdot \text{width})$$

will fit within the execution time limit.

Example

Input:

$$\text{paragraphs} = \begin{bmatrix} \{\text{"hello", "world"}\}, \\ \{\text{"How", "areYou", "doing"}\}, \\ \{\text{"Please", "look", "and", "align", "to", "center"}\} \end{bmatrix}, \quad \text{width} = 16$$

Approach:

1. Line Construction:

- For each paragraph, words are added sequentially to the current line.
- Words are separated by a single space.
- If adding a word exceeds **width**, a new line is started.

2. Center Alignment:

- After forming a line, if its length is less than **width**, spaces are added before and after the text to center it.
- Any extra space (when padding is odd) is added to the right.

3. Border Addition:

- A top and bottom border of asterisks is added.
- Each formatted line is enclosed between ***** and *****.

Output:

```
[
"*****",
"*  hello world  *",
"* How areYou doing *",
"*  Please look  *",
"*   and align   *",
"*   to center   *",
"*****"
]
```

10. Reconstruct Travel Journey

Description

A traveler visited a series of unique landmarks on a journey. Unfortunately, their travel journal was damaged, and they can no longer remember the exact order of their visits. However, they do have a collection of photos, each showing exactly two landmarks that were visited consecutively (either landmark could have been visited first).

You are given the collection of photos represented as pairs of landmark IDs in **travelPhotos**. Using this information, help the traveler reconstruct the complete journey.

Each landmark was visited exactly once, and for every consecutive pair of landmarks in the journey, there exists a photo containing both landmarks.

You may reconstruct the journey in either forward or reverse order — both are considered correct.

Task

Given **travelPhotos**, return an array representing the order in which the landmarks were visited.

Input

- **travelPhotos**: An array of pairs, where each pair contains two integers representing landmarks that were visited consecutively.

Constraints

- $1 \leq \text{travelPhotos.length} \leq 5 \times 10^4$
- $\text{travelPhotos}[i].length = 2$
- $1 \leq \text{travelPhotos}[i][j] \leq 10^6$

Example

Input:

$$\text{travelPhotos} = \begin{bmatrix} [3, 5], \\ [1, 4], \\ [2, 4], \\ [1, 5] \end{bmatrix}$$

Explanation

- The photos show that landmarks 3 and 5 were visited consecutively.
- Similarly, landmarks 1 and 4, 2 and 4, and 1 and 5 were also consecutive.
- By analyzing these connections, we can reconstruct one valid journey as:

$$[3, 5, 1, 4, 2]$$

- The reverse journey:

$$[2, 4, 1, 5, 3]$$

is also a valid answer.

11. Harmonious Building Structures

Description

You are managing a row of building structures with varying heights, represented by an array of integers **structures**. You want to transform these structures into a harmonious pattern where the height of each structure differs from its adjacent neighbors by exactly one unit.

The final pattern must form either:

- a strictly ascending sequence (each next structure is taller by exactly 1), or
- a strictly descending sequence (each next structure is shorter by exactly 1).

To modify the structures, you are allowed to perform only the following operation:

- In a single operation, you may increase the height of any one structure by exactly 1 unit.

Your goal is to determine the **minimum number of operations** required to transform the given structures into either a valid ascending or descending stepwise pattern. If both patterns are possible, choose the one that requires fewer operations.

Task

Given the array **structures**, return the minimum number of operations required to achieve a harmonious pattern.

Constraints

- $1 \leq \text{structures.length} \leq 10^5$
- $1 \leq \text{structures}[i] \leq 10^9$

Example

Input:

structures = [1, 4, 3, 2]

Explanation: The optimal approach is to form a descending stepwise pattern.

- Add 4 units to the first structure.

The final heights become:

[5, 4, 3, 2]

This sequence is strictly descending with a difference of exactly 1 between adjacent structures.

Output:

solution(structures) = 4

Input:

structures = [5, 7, 9, 4, 11]

Explanation: The optimal approach is again a descending pattern.

- Add 2 units to the first structure ($5 \rightarrow 7$).
- Add 1 unit to the second structure ($7 \rightarrow 8$).
- Add 0 units to the remaining structures.

The final heights become:

[7, 8, 9, 10, 11]

This forms a valid ascending stepwise pattern.

Output:

solution(structures) = 9

Willings INC

1. Longest Palindromic Substring

Description

Given a string s , return the **longest palindromic substring** in s .

Constraints

- $1 \leq |s| \leq 10^5$
- s consists of only digits and English letters.

Example

Assumptions: $s = \text{"babad"}$.

Approach:

- The substring **"bab"** is a palindrome.
- The substring **"aba"** is also a valid palindrome of the same length.

Either answer is acceptable.

Sample Case 1

Sample Input:

babad

Sample Output:

bab

Cisco

1. Word Search

Problem Description

You are given an $m \times n$ board of characters and a list of strings `words`. Your task is to find all words from the list that can be constructed using the characters on the board.

Each word must be formed by sequentially adjacent cells, where adjacent cells are defined as those that are horizontally or vertically neighboring. The same cell may not be used more than once in a single word.

Return all words from the given list that can be found on the board.

Examples

Example 1:

- **Input:**

$$\text{board} = \begin{bmatrix} o & a & a & n \\ e & t & a & e \\ i & h & k & r \\ i & f & l & v \end{bmatrix}$$

`words = ["oath", "pea", "eat", "rain"]`

- **Output:**

`["eat", "oath"]`

Example 2:

- **Input:**

$$\text{board} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

`words = ["abcb"]`

- **Output:**

`[]`

Constraints

- $m == \text{board.length}$
- $n == \text{board}[i].\text{length}$
- $1 \leq m, n \leq 12$
- `board[i][j]` is a lowercase English letter
- $1 \leq \text{words.length} \leq 3 \times 10^4$
- $1 \leq \text{words}[i].\text{length} \leq 10$

- `words[i]` consists of lowercase English letters
- All strings in `words` are unique

2. Seed Influence Maximization

Description

Your team at a leading technology company is launching a cutting-edge networking product. To generate buzz, the company plans to run a targeted social media campaign by selecting a set of influential users (called *seed users*) from a professional network.

Each user in the network has:

- A **market value**, representing how valuable it is for the campaign to reach that user.
- (Optionally) a **cost** if the user can be chosen as a seed.

Influence spreads through the network starting from the chosen seeds to their direct friends, then friends-of-friends, and so on, up to a specified maximum depth d (degrees of separation). A user contributes their market value **at most once**, even if reached through multiple paths.

The company has a limited marketing budget and cannot afford to select all potential seeds. A brute-force approach that checks all possible seed combinations is computationally infeasible.

Task

Design an algorithm to select an **optimal set of seed users** such that:

- The total cost of selected seeds does not exceed the given budget.
- The total market value of **all unique users reached** (within depth d from the seeds) is maximized.

The algorithm should make intelligent choices by considering both:

- The cost-effectiveness of a seed.
- The overlap in influence between multiple seeds.

Note

- Only users listed as potential seeds can be selected as seeds.
- Influence propagation is based on an undirected network.
- A user's value is counted only once, even if influenced multiple times.
- The solution is expected to be heuristic or greedy in nature, as exact optimization is infeasible.

Constraints

- $1 \leq n \leq 10^5$ users
- $0 \leq m \leq 2 \times 10^5$ connections
- $1 \leq \text{number of potential seeds} \leq n$
- Seed costs and user values are positive integers

- $1 \leq \text{budget} \leq 10^9$
- $1 \leq d \leq 10$

Example

Network Connections:

$(1, 2), (1, 3), (4, 5)$

Potential Seeds and Costs:

$1 \rightarrow 50, \quad 4 \rightarrow 60$

User Market Values:

$[10, 20, 30, 5, 5]$

Budget: 100

Depth: 1

Explanation:

- Choosing User 1 as a seed costs 50 and reaches users $\{1, 2, 3\}$. Total value = $10 + 20 + 30 = 60$.
- Choosing User 4 as a seed costs 60 and reaches users $\{4, 5\}$. Total value = $5 + 5 = 10$.

User 1 provides a higher value per unit cost. After selecting User 1, the remaining budget is insufficient to add another beneficial seed.

Output:

- Selected Seeds: $\{1\}$
- Total Market Value Reached: 60
- Total Cost: 50

Trilogy Innovations

1. Cryptarithm Solver

Description

A **cryptarithm** is a mathematical puzzle where letters are substituted with digits such that a given arithmetic equation holds true. Each letter represents a unique digit from 0 to 9, and no two letters may represent the same digit.

You are given a cryptarithm represented as an array of three strings **crypt**. Each string consists only of uppercase English letters and represents a word interpreted as a number after substituting letters with digits.

The cryptarithm has the following structure:

$$\text{word1} + \text{word2} = \text{word3}$$

A solution is considered **valid** if:

- Each letter maps to a distinct digit.
- No multi-digit number has a leading zero.
- The arithmetic equation holds true after substitution.

Task

Given the array **crypt**, count the total number of valid solutions that satisfy the cryptarithm.

Note

- The same digit cannot be assigned to more than one letter.
- Leading zeros are not allowed for any word with more than one letter.

Constraints

- **crypt** contains exactly 3 strings.
- Each string contains only uppercase English letters.
- $1 \leq \text{crypt}[i].\text{length} \leq 35$

Examples

Example 1

Input:

crypt = ["SEND", "MORE", "MONEY"]

Output:

1

Explanation: There is exactly one valid solution:

$$9567 + 1085 = 10652$$

with the mapping:

$$S = 9, E = 5, N = 6, D = 7, M = 1, O = 0, R = 8, Y = 2$$

Example 2

Input:

crypt = ["GREEN", "BLUE", "BLACK"]

Output:

12

Explanation: There are 12 distinct digit assignments that satisfy the equation GREEN + BLUE = BLACK. Some valid solutions include:

$$54889 + 6138 = 61027$$

$$18559 + 2075 = 20634$$

$$72449 + 8064 = 80513$$

$$48229 + 5372 = 53601$$

$$47119 + 5261 = 52380$$

...

2. Fill the First Column

Description

You are given a rectangular board divided into a uniform grid of square cells. Some cells of the board are already occupied with blocks, while others are empty. Blocks are denoted by the character #, and empty cells are denoted by ..

You can add new blocks to the board using the following process:

- First, you choose a row index.
- A new block is thrown into the chosen row from the left side.
- The block initially appears in the leftmost cell of that row and moves to the right until it encounters another block or reaches the end of the row.
- Once the block can no longer move to the right, it starts falling down until it encounters another block or reaches the last row of the board.

Your objective is to repeatedly add blocks so that the **entire first column** of the board becomes filled with blocks.

Task

Given the initial configuration of the board, calculate:

- the **minimum** number of moves required to fill the first column with blocks, and
- the **maximum** number of moves required to fill the first column with blocks.

Return these two values as an array `[minMoves, maxMoves]`.

Constraints

- $1 \leq \text{field.length} \leq 12$
- $1 \leq \text{field}[0].\text{length} \leq 12$

Example

Input:

$$\text{field} = \begin{bmatrix} . & . & \# & . \\ . & \# & . & . \\ \# & . & . & . \end{bmatrix}$$

Output:

$$\text{solution}(\text{field}) = [3, 6]$$

Explanation

- **Minimum moves:** To minimize the number of moves, we repeatedly choose the first row. After three moves, blocks settle in such a way that the first column becomes completely filled.
- **Maximum moves:** To maximize the number of moves, we first choose the third row three times, then the second row twice, and finally the first row once. This results in six total moves before the first column is fully filled.

3. Almost Matching Substrings

Description

You are creating a new programming language with an enhanced string comparison feature. While most programming languages can check if two strings are exactly matching, your language introduces the concept of *almost matching* strings.

Two strings are said to be **almost matching** if:

- They are of equal length, and
- All corresponding characters are the same *except for exactly one position*.

For example, the strings "cat" and "bat" are almost matching, whereas "cat" and "dog" are not.

To efficiently test this feature, instead of comparing arbitrary strings, you are given a single string **s** and an integer **k**. Your task is to count the number of pairs of substrings of **s** that are almost matching and differ at exactly the **k**-th character (0-based index).

It is guaranteed that both substrings must have length strictly greater than **k**, otherwise they would not have a **k**-th character.

Substrings are identified by their indices in the original string, so two substrings with the same content but different positions are considered distinct.

Task

Given a string **s** and an integer **k**, count the number of pairs of substrings of **s** that:

- Have equal length,
- Differ at exactly one character, and
- Differ specifically at index **k**.

Note

- Indices are 0-based.
- Multiple occurrences of the same substring content are counted separately if they come from different indices.
- Only pairs of substrings with length greater than **k** are valid.

Constraints

- $1 \leq |s| \leq 10^5$
- $0 \leq k < |s|$
- **s** consists of lowercase English letters only

Example

Input:

s = "abacaba", **k** = 1

Output:

solution(s, k) = 8

Explanation

Using (i, j) to represent the start and end indices of the first substring, and (l, m) for the second substring, the 8 valid pairs of almost matching substrings are:

- ("aba", "aca") with $(i, j) = (0, 2)$ and $(l, m) = (2, 4)$
- ("aba", "aca") with $(i, j) = (4, 6)$ and $(l, m) = (2, 4)$
- ("aca", "aba") with $(i, j) = (2, 4)$ and $(l, m) = (0, 2)$
- ("aca", "aba") with $(i, j) = (2, 4)$ and $(l, m) = (4, 6)$
- ("ac", "ab") with $(i, j) = (2, 3)$ and $(l, m) = (0, 1)$
- ("ac", "ab") with $(i, j) = (2, 3)$ and $(l, m) = (4, 5)$
- ("ab", "ac") with $(i, j) = (0, 1)$ and $(l, m) = (2, 3)$
- ("ab", "ac") with $(i, j) = (4, 5)$ and $(l, m) = (2, 3)$

Each pair differs at exactly index **k** = 1 and matches at all other positions.

4. Mario and the River Platforms

Description

Welcome to the magical world of Mario's adventure. Mario starts on platform 0 and needs to reach platform $A + 1$. Unfortunately, a wide river of width A blocks his way.

Mario cannot walk on water, but he has a special ability: he can jump up to B units at a time, either forward or backward.

On the river, there are several wooden platforms floating, similar to those seen in Mario's games. In total, there are M platforms, and their sizes are given in an integer array $\mathbf{C}$. Mario is allowed to move these platforms before starting his journey, but with the following restrictions:

- The relative order of the platforms cannot be changed.
- Platforms cannot overlap.
- All platform movements must be completed before Mario starts jumping.

Your task is to help Mario cross the river and reach platform $A + 1$.

If it is impossible for Mario to reach platform $A + 1$, return a vector containing a single element -1 . Otherwise, return a vector of size A representing the river cells.

Each cell in the returned vector should be defined as follows:

- If the i^{th} cell does not belong to any platform, its value should be 0.
- Otherwise, its value should be the index (1-based) of the platform to which the cell belongs.

If multiple valid solutions exist, return the **lexicographically smallest** one.

Task

Given integers A , B , and an array $\mathbf{C}$, determine whether Mario can reach platform $A + 1$. If possible, return the lexicographically smallest valid configuration of river cells; otherwise, return $[-1]$.

Constraints

- $1 \leq A \leq 10^5$
- $1 \leq B \leq 10^5$
- $1 \leq \text{length of } C \leq 10^5$
- $1 \leq C[i] \leq 10^5$

Examples

Example 1

Input:

$$A = 7, \quad B = 2, \quad C = [1, 2, 1]$$

Output:

$$[0, 1, 0, 2, 2, 0, 3]$$

Explanation: The lexicographically smallest valid configuration of the river cells is:

$$[0, 1, 0, 2, 2, 0, 3]$$

Mario performs the sequence of jumps:

$$0 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 8$$

and successfully reaches platform $A + 1 = 8$.

Example 2

Input:

$$A = 10, \quad B = 5, \quad C = [2]$$

Output:

$$[0, 0, 0, 1, 1, 0, 0, 0, 0, 0]$$

Explanation: The lexicographically smallest valid configuration is:

$$[0, 0, 0, 1, 1, 0, 0, 0, 0, 0]$$

Mario jumps as follows:

$$0 \rightarrow 5 \rightarrow 6 \rightarrow 11$$

and reaches platform $A + 1 = 11$.

HSBC

1. Connected Components Product

Description

You are given a two-dimensional grid consisting of cells that are either occupied or empty. Two occupied cells are considered **connected** if they share a common side, i.e., they are adjacent in one of the four directions:

up, down, left, right.

A **connected component** is defined as a maximal set of occupied cells such that any two cells in the set can be connected by a path of adjacent occupied cells using only these four directions.

Task

Your task is to analyze the grid and determine:

- a : the number of connected components of size exactly 1
- b : the number of connected components of size exactly 2
- c : the number of connected components of size exactly 3

After computing these values, return the result:

$$a \times b^c$$

Constraints

- $1 \leq \text{rows}, \text{cols} \leq 10^3$
- Each cell of the grid is either occupied or empty

Example

Input:

$$\text{grid} = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

Here, 1 represents an occupied cell and 0 represents an empty cell.

Explanation

The connected components are:

- Three components of size 1
- One component of size 2
- One component of size 3

Thus:

$$a = 3, \quad b = 1, \quad c = 1$$

Output

$$a \times b^c = 3 \times 1^1 = 3$$

2. Word Censorship

Description

You are given a dictionary containing a set of words that must be censored from a given text. Whenever a word from the dictionary appears in the text, it should be replaced with a sequence of characters (.) of the same length as the word.

The replacement must be applied to *all occurrences* of dictionary words in the text.

Task

Given:

- a string **text**, and
- a list of strings **dictionary**

replace every occurrence of each word in **dictionary** within **text** by a string of `.` characters of equal length.

Return the modified text after all replacements have been performed.

Constraints

- $1 \leq |\text{text}| \leq 10^5$
- $1 \leq |\text{dictionary}| \leq 10^5$
- $1 \leq |\text{dictionary}[i]| \leq 100$
- All strings consist only of lowercase English letters

Example

Input:

```
text = "thisisabadexample"
dictionary = ["bad", "example"]
```

Explanation

The word "bad" is replaced by "..." and the word "example" is replaced by ".....".

Output:

```
"thisisa....."
```

3. Count Adjacent Equal Characters

Description

You are given a string **s** consisting of lowercase English letters.

Your task is to count the number of characters that are equal to the character immediately preceding them in the string.

In other words, count the number of indices i such that:

$$s[i] = s[i - 1], \quad 1 \leq i < |s|$$

Task

Given the string **s**, return the total number of adjacent equal characters.

Constraints

- $1 \leq |s| \leq 10^5$
- **s** consists only of lowercase English letters

Example

Input:

"aabb"

Explanation:

- $s[1] = s[0] = \text{'a'}$
- $s[3] = s[2] = \text{'b'}$

Total count = **2**

Input:

"aaabb"

Explanation:

- $s[1] = s[0] = \text{'a'}$
- $s[2] = s[1] = \text{'a'}$
- $s[4] = s[3] = \text{'b'}$

Total count = **3**

(Bonus) Remove K Digits

Description

You are given a string **num** representing a non-negative integer, and an integer **k**. Your task is to remove exactly **k** digits from **num** such that the resulting number is the **smallest possible**.

The relative order of the remaining digits must be preserved.

Task

Given the string **num** and the integer **k**, return the smallest possible integer (as a string) after removing exactly **k** digits.

Notes

- The result should not contain leading zeros.
- If all digits are removed, return `"0"`.
- The input number may be very large, so it must be processed as a string.

Constraints

- $1 \leq k \leq |\text{num}| \leq 10^5$
- **num** consists only of digits (`'0'` to `'9'`)
- **num** does not contain leading zeros unless it is exactly `"0"`

Example

Input:

`num = "1432219", k = 3`

Explanation: Removing the digits **4**, **3**, and **2** results in the number `"1219"`, which is the smallest possible value obtainable after removing exactly three digits.

Output:

`"1219"`

Samsung

1. Divisible by Powers of Two

Description

You are given a string **s** consisting only of decimal digits (**0–9**). The length of the string can be very large (up to 10^5), so the integer value represented by the string or any of its substrings may be far too large to store or process directly using standard numeric types.

Consider all powers of two of the form:

$$2^k \quad \text{where } 0 \leq k < 800.$$

Each power of two is written in its usual decimal representation (without leading zeros).

Task

Your task is to count how many substrings of **s** are exactly divisible by the decimal representation of *some* power of two 2^k with $0 \leq k < 800$.

Note

- Substrings must not contain leading zeros.
- Substrings are distinguished by their positions, not just by their values.

Constraints

- $1 \leq |s| \leq 10^5$
- **s** consists only of characters '**0**' through '**9**'
- $0 \leq k < 800$

Example

Input:

s = "204800"

Explanation: Valid substrings of **s** that match a power of two:

- "2048" matches 2^{11}

Substrings such as "048" or "00" are *not* considered valid because they either have leading zeros or do not match the decimal representation of any power of two.

Output:

Total number of valid substrings = 1

InMobi Groups Pvt Ltd

1. Range Maximum Query with Updates

Description

You are given an integer array A of size N . You need to process Q queries of the following two types:

- **Update Query:** Update the value at a given index.
- **Range Query:** Find the maximum value in a given subarray.

Query Types

Each query is of one of the following forms:

- **1 i x** — Set $A[i] = x$
- **2 l r** — Output the maximum value in the range $[l, r]$

Task

Process all queries and print the result for each range query.

Constraints

- $1 \leq N, Q \leq 10^5$
- $1 \leq A[i], x \leq 10^9$
- $0 \leq i < N$
- $0 \leq l \leq r < N$

Example

Input:

```
N = 5
A = [1, 3, 5, 7, 9]
Q = 2
Queries:
2 1 3
1 2 10
```

Explanation:

- Query **2 1 3**: Maximum in range $[1,3]$ is **7**
- Query **1 2 10**: Update index 2 to 10

Output:

```
7
```

Intel

1. Minimum Rotations to Match Strings

Description

You are given two strings **s1** and **s2** of equal length. Both strings consist of the same set of characters with the same frequency, but their arrangements may be different.

In one rotation operation, you may perform a **left rotation** on **s2**, where the first character is moved to the end of the string.

Task

Determine the **minimum number of rotations** required to transform **s2** into **s1**. If it is not possible to transform **s2** into **s1** using rotations, return **-1**.

Notes

- A left rotation by one position shifts all characters one place to the left.
- Rotations must preserve the relative order of characters.

Constraints

- $1 \leq |s1| = |s2| \leq 100$
- **s1** and **s2** consist of lowercase English letters
- **s1** and **s2** contain the same characters with the same frequencies

Example

Input:

- **s1** = "abcd"
- **s2** = "cdab"

Explanation: Performing left rotations on **s2**:

- Rotation 1: "dabc"
- Rotation 2: "abcd"

After 2 rotations, **s2** becomes equal to **s1**.

Output:

2

2. Convert and Reverse String

Description

You are given a string s consisting of lowercase English letters.

You need to perform the following operations on the string in the given order:

1. Replace every occurrence of the character **p** with **i**.
2. Replace every occurrence of the character **k** with **a**.
3. Reverse the resulting string.

Task

Return the final string obtained after performing all the above operations.

Constraints

- $1 \leq |s| \leq 10^5$
- The string s contains only lowercase English letters

Example

Input:

$s = \text{"pikkap"}$

Explanation:

- Replace **p** with **i**: "iikkai"
- Replace **k** with **a**: "iaaaai"
- Reverse the string: "iaaaaii"

Output:

"iaaaaii"

Amazon

1. Maximum Equal Frequency Partitions

Description

You are given a string s of length n . For every prefix $s[0 \dots i]$, determine the maximum number of consecutive partitions that can be formed such that each partition is valid. A substring is considered valid if all distinct characters present in that substring occur with the same frequency. The partitions must be contiguous, non-overlapping, and together cover the entire prefix. Return an array arr of size n where

$$arr[i] = \text{maximum number of valid partitions possible for prefix } s[0 \dots i].$$

Constraints

- $1 \leq n \leq 10^5$
- s consists only of lowercase English letters

Example 1

Input:

$s = \text{"abcbabcab"}$

Explanation:

For the prefixes "a", "ab", "abc", "abca", and "abcab", only one valid partition is possible. For the prefix "abcbabc", the string can be divided into two valid consecutive partitions:

$(\text{"abc"}, \text{"abc"})$

For the prefixes "abcbabca" and "abcbabcab", only one valid partition is possible. For the prefix "abcbabcabc", the maximum number of valid partitions is:

$(\text{"abc"}, \text{"abc"}, \text{"abc"})$

which gives 3 partitions.

Output:

$[1, 1, 1, 1, 1, 2, 1, 1, 3]$

Sample Case 1

Sample Input:

$s = \text{"abcbabcab"}$

Sample Output:

$[1, 1, 1, 1, 1, 2, 1, 1, 3]$

Sample Case 2

Sample Input:

`s = "abaabb"`

Sample Output:

`[1,1,1,1,1,1]`

2. Array Deletion Process

Description

You are given an integer vector arr of size n . In each iteration, find the minimum element in the current array. Suppose its index is i . Remove the i^{th} element along with its adjacent neighbors, namely the $(i - 1)^{th}$ and $(i + 1)^{th}$ elements if they exist. Continue this process until the array becomes empty. Your task is to determine the number of iterations required to completely remove all elements from the array.

Constraints

- $1 \leq n \leq 10^3$
- $-10^9 \leq arr[i] \leq 10^9$

Example 1

Input:

$arr = [4, 2, 7, 1, 5]$

Explanation:

Initially, the minimum element is 1 at index 3. Removing it along with its neighbors removes the elements

$7, 1, 5$

and the remaining array becomes

$[4, 2]$.

In the next iteration, the minimum element is 2, and removing it along with its neighbor removes all remaining elements. Therefore, the array becomes empty in 2 iterations.

Output:

2

Sample Case 1

Sample Input:

`arr = [4, 2, 7, 1, 5]`

Sample Output:

2

3. Maximum Revenue from VM Stocks

Problem Statement

You are given an integer array *vmstock* representing available VM stock units, where each element denotes the number of units in a stock type. There are *m* customers, and for each customer you must first find the smallest non-zero element in the array and the largest element in the array. The customer pays a cost equal to:

$$cost = \min_non_zero + \max_element.$$

After the transaction, decrease the maximum element by 1. Continue this process for all *m* customers and return the total revenue collected after serving all customers.

Constraints

- $1 \leq |vmstock| \leq 10^5$
- $1 \leq vmstock[i] \leq 10^9$
- $1 \leq m \leq 10^5$

Example 1

Input:

$$vmstock = [1, 2, 4], \quad m = 1$$

Explanation:

The minimum non-zero element is 1 and the maximum element is 4. Therefore, the revenue collected from the customer is:

$$1 + 4 = 5.$$

After the transaction, the maximum element is decreased by 1, so the updated array becomes:

$$[1, 2, 3].$$

Output:

5

Sample Case 1

Sample Input:

vmstock = [1, 2, 4]

m = 1

Sample Output:

5

Note

If all elements become zero before serving all customers, then the remaining customers contribute 0 revenue.

4. Prefix Balance Frequency

Problem Statement

You are given a string s consisting of uppercase English letters. For every prefix of the string, compute the frequency of each character appearing in that prefix. If all characters in the prefix have the same frequency, output that frequency; otherwise, output 1.

Return an array $result$ where:

$$result[i]$$

represents the answer for the prefix:

$$s[0 \dots i].$$

Constraints

- $1 \leq |s| \leq 10^5$
- $s[i]$ is an uppercase English letter

Example 1

Input:

$$s = \text{"ABAB"}$$

Output:

$$[1, 1, 1, 2]$$

Explanation:

For the prefix “A”, the frequency of character A is 1, so the answer is 1. For the prefix “AB”, both characters occur once, so the answer remains 1. For the prefix “ABA”, the frequencies become:

$$A : 2, \quad B : 1,$$

which are not equal, so the answer is 1. For the prefix “ABAB”, both characters occur twice:

$$A : 2, \quad B : 2,$$

therefore the answer for this prefix is 2.

Sample Case 1

Sample Input:

$s = \text{"ABAB"}$

Sample Output:

$[1, 1, 1, 2]$